Algoritmos y Estructuras de Datos I · II Semestre 2026

Ordenar y buscar sin perder tiempo

Radix Sort · Búsqueda binaria · Búsqueda por hash

Marco Chavarría León · Santiago Portuguez Hernández · Francisco Ramírez Delgado
Jan Carlo Vargas Araya · Carlos Zhou Fernández

Instituto Tecnológico de Costa Rica · Escuela de Ingeniería en Computadores

EL PROBLEMA

¿Cómo encuentras un carné entre 1 000 000?

UNO POR UNO

1 000 000

comparaciones en el peor caso

ORDENAR Y PARTIR A LA MITAD

≈ 20

comparaciones, descartando la mitad cada vez

IR DIRECTO AL CASILLERO

≈ 1

acceso en promedio

Hoy: un algoritmo para **ordenar** y dos para **buscar**.

Ordenar sin comparar: dígito por dígito

Reparte las cartas en 10 casilleros según su último dígito, recógelas en orden y repite con el dígito siguiente.

Pasada 1

Unidades

170 → 0 · 045 → 5 · 802 → 2

Pasada 2

Decenas

170 → 7 · 045 → 4 · 802 → 0

Pasada 3

Centenas

170 → 1 · 045 → 0 · 802 → 8

Ejemplo de hoy: [170, 45, 75, 90, 802, 24, 2, 66]. El mayor tiene 3 dígitos, así que son 3 pasadas.

Tres pasadas, tres dígitos

Pasada 1 de 3 · dígito de las unidades



066 → su dígito es 6 → va al casillero 6

Cada pasada es un counting sort estable

1 · Contar
cuántos terminan en cada dígito

`[2, 0, 2, 0, 1, 2, 1, 0, 0, 0]`

2 · Acumular
dónde termina cada casillero

`[2, 2, 4, 4, 5, 7, 8, 8, 8, 8]`

3 · Colocar desde el final
resultado de la pasada 1

`[170, 90, 802, 2, 24, 45, 75, 66]`

Estable = los empates conservan su orden. 802 y 2 empatan en 2: 802 venía primero y sigue primero.

Sin optimizar: base 10 y enteros ≥ 0

```python
def radix_sort(datos):
    datos = list(datos)
    if not datos:
        return datos
    exp = 1
    while max(datos) // exp > 0:
        datos = _ordenar_por_digito(datos, exp)
        exp *= 10
    return datos
```

```
>>> radix_sort([170, 45, 75, 90, 802, 24, 2, 66])
[2, 24, 45, 66, 75, 90, 170, 802]
```

```python
def _ordenar_por_digito(datos, exp):
    cuenta = [0] * 10
    for x in datos:
        cuenta[(x // exp) % 10] += 1
    for d in range(1, 10):
        cuenta[d] += cuenta[d - 1]
    salida = [0] * len(datos)
    for x in reversed(datos):
        d = (x // exp) % 10
        cuenta[d] -= 1
        salida[cuenta[d]] = x
    return salida
```

Siempre d pasadas, sin importar el orden

Caso	Tiempo	Por qué
Mejor	O(d·(n+b))	Siempre hace d pasadas
Promedio	O(d·(n+b))	No depende del orden inicial
Peor	O(d·(n+b))	Igual: d pasadas de n+b trabajo
Espacio	O(n+b)	Lista de salida (n) y cuenta (b)

n = cantidad de datos

d = dígitos del número mayor

b = base (10 casilleros)

Con n = 8, d = 3, b = 10: 3 × (8 + 10) = 54 pasos

Si d es fijo, el tiempo es lineal: O(n).

Rápido con enteros, limitado a ellos

Ventajas

- No compara números entre sí
- Lineal si d es fijo
- Estable: respeta los empates

Desventajas

- Solo claves enteras o de largo fijo
- Memoria extra: O(n+b)
- Con d enorme deja de ser rápido

Casos de uso

- Carnés e identificaciones
- Fechas y códigos postales
- Preparar datos para buscar

Ya está ordenado. ¿Cómo lo buscamos rápido? → **Búsqueda binaria**

Adivina un número del 1 al 100

Cada pregunta («¿es mayor o menor?») descarta la mitad de las opciones que quedan.



7 preguntas bastan para 100 opciones. 20 para un millón. 30 para mil millones.

Condición: la lista tiene que estar ordenada. Por eso Radix va primero.

Una búsqueda que acierta y otra que no

Buscamos: 23

Comparaciones: 3

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
2	5	8	12	16	23	38	56	72	91
					bajo · medio	alto			

¡Encontrado! 23 está en el índice 5, con 3 comparaciones.

Once líneas, tres variables

```python
def busqueda_binaria(ordenada, objetivo):
    bajo, alto = 0, len(ordenada) - 1
    while bajo ⩽ alto:
        medio = (bajo + alto) // 2
        if ordenada[medio] == objetivo:
            return medio
        if ordenada[medio] < objetivo:
            bajo = medio + 1
        else:
            alto = medio - 1
    return -1
```

```
>>> busqueda_binaria(orden, 23)
5
```

Traza buscando 23 en 2, 5, 8, 12, 16, 23, 38, 56, 72, 91

bajo	alto	medio	dato
0	9	4	16
5	9	7	56
5	6	5	23

`16 < 23 → bajo = 5`
`56 > 23 → alto = 6`
`23 = 23 → ¡índice 5!`

Cada paso descarta la mitad

Caso	Tiempo	Por qué
Mejor	O(1)	El dato está justo en el medio
Promedio	O(log n)	Cada paso descarta la mitad
Peor	O(log n)	No está, o está en un extremo
Espacio	O(1)	Solo guarda bajo, alto y medio

10 datos → máximo **4** comparaciones
1 000 datos → máximo **10**
1 000 000 datos → máximo **20**
1 000 000 000 datos → máximo **30**

Ojo: buscar rápido exige ordenar antes.

Ordenar con Radix se paga una vez; después cada búsqueda cuesta O(log n).

Muy rápida, pero exige orden

Ventajas

- Muy rápida: crece como log n
- Memoria constante: O(1)
- Fácil de programar

Desventajas

- Exige datos ordenados
- Necesita acceso por índice
- Insertar y mantener el orden es caro

Casos de uso

- Diccionarios y listas telefónicas
- Bases de datos ordenadas
- Buscar un punto de corte

¿Y si no queremos ordenar? → **Búsqueda por hash**

La clave dice dónde guardarla

Como un hotel con casilleros numerados: la función hash convierte la clave en el número de casillero.

Clave

36

el carné que queremos guardar

→

Función hash, con m = 7

36 mod 7 = 1

h(k) = k mod m: el residuo de dividir

→

Casillero

1

ahí se guarda y ahí se busca

Para buscar 36 no revisamos todo: vamos directo al casillero 1.

Dos claves, un casillero: colisión

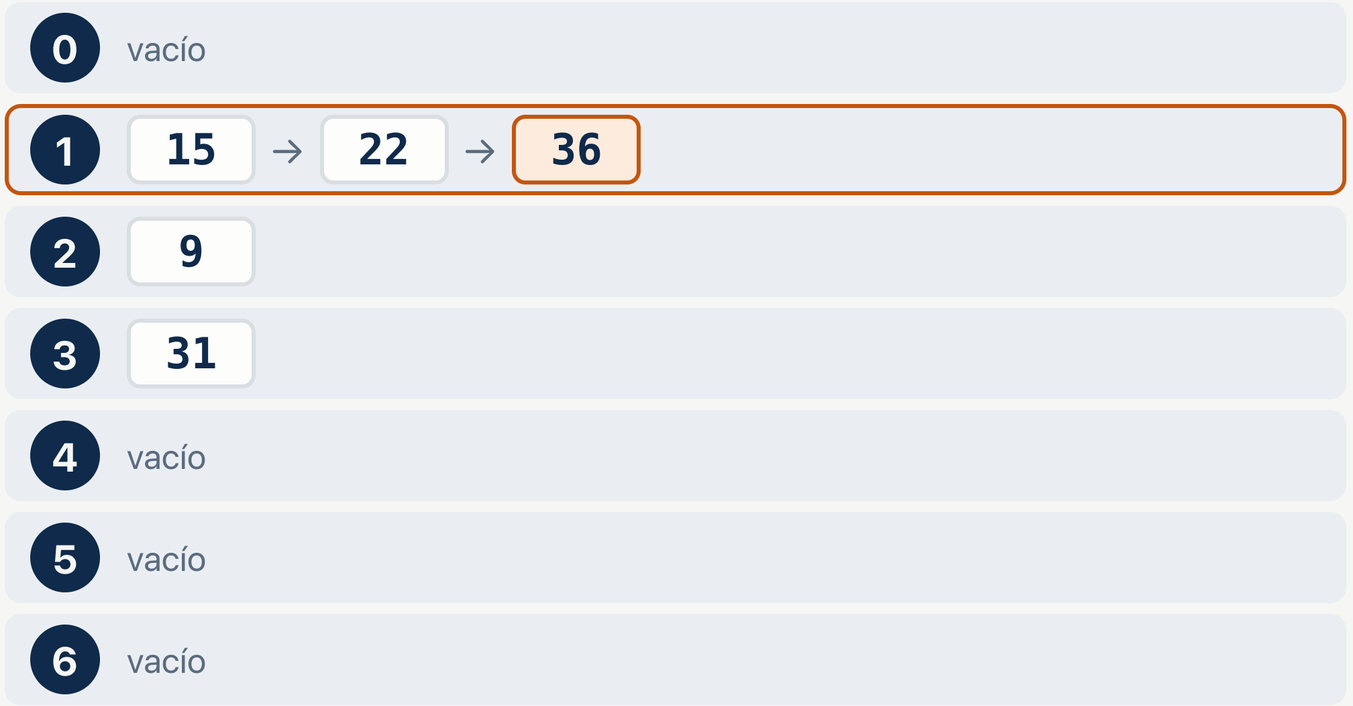
Clave	Cálculo	Casillero
15	15 mod 7	1
22	22 mod 7	1
9	9 mod 7	2
31	31 mod 7	3
36	36 mod 7	1

15, 22 y 36 caen en el casillero 1.

Casillero	Contenido (lista)
0	vacío
1	**15 → 22 → 36**
2	9
3	31
4	vacío
5	vacío
6	vacío

Encadenamiento: cada casillero guarda una lista con todas las claves que caen ahí.

Insertar, colisionar y buscar



Insertando 36

36 mod 7 = 1

Colisión: el casillero 1 ya tenía 2 claves. Se agrega al final de su lista.

Sin optimizar: m fijo, sin rehashing

```python
class TablaHash:
    def __init__(self, m=7):
        self.m = m
        self.casillas = [[] for _ in range(m)]

    def _h(self, clave):
        return clave % self.m

    def insertar(self, clave):
        self.casillas[self._h(clave)].append(clave)

    def buscar(self, clave):
        return clave in self.casillas[self._h(clave)]
```

```python
t = TablaHash(7)
for k in [15, 22, 9, 31, 36]:
    t.insertar(k)
t.buscar(36)
t.buscar(99)
t.casillas
```

```
True
False
[[], [15, 22, 36], [9], [31], [], [], []]
```

Casi instantánea, si las listas son cortas

Caso	Tiempo	Por qué
Mejor	O(1)	La clave está sola en su casillero
Promedio	O(1 + α)	α = n/m es el largo medio de la lista
Peor	O(n)	Todas las claves caen en un casillero
Espacio	O(n+m)	n claves y m casilleros

Nuestro ejemplo: α = 5 / 7 ≈ 0,71

Menos de una clave por casillero, en promedio.

Peor caso: 15, 22, 8, 29 y 36

Todas caen en el casillero 1: buscar 36 cuesta 5 comparaciones.

Velocidad a cambio de orden y memoria

Ventajas

- Búsqueda promedio casi instantánea
- No necesita datos ordenados
- Insertar y buscar usan el mismo cálculo

Desventajas

- Colisiones: el peor caso es O(n)
- No guarda el orden
- Con m fijo, si se llena se vuelve lenta

Casos de uso

- Cachés y diccionarios (dict de Python)
- Índices de bases de datos
- Tablas de símbolos en compiladores

Tres herramientas, tres trabajos. → **¿Cuál elegir?**

COMPARACIÓN

¿Cuál uso y cuándo?

	Radix Sort	Búsqueda binaria	Búsqueda por hash
Sirve para	Ordenar enteros	Buscar en lo ordenado	Buscar sin ordenar
Tiempo típico	O(d·(n+b))	O(log n)	O(1) promedio
Peor caso	O(d·(n+b))	O(log n)	O(n)
Necesita	Claves enteras	Lista ordenada	Buena función hash
Memoria extra	O(n+b)	O(1)	O(n+m)

Pocas búsquedas: ordena una vez y usa binaria.
Muchas búsquedas, el orden no importa: hash.
Enteros que hay que ordenar: Radix Sort.

CONCLUSIONES

La forma de los datos decide el algoritmo

1

Radix ordena sin comparar

d pasadas estables; lineal si d es fijo.

2

La binaria gana con orden

O(log n): 20 pasos para un millón.

3

El hash cambia orden y memoria por velocidad

O(1) promedio; con colisiones, O(n).

REPOSITORIO Y REFERENCIAS

Todo el código está en GitHub

Repositorio del grupo

`github.com/Saportu/Desarrollo-e-investigaci-n-de-Algoritmos-de-Ordenamiento-y-B-squeda`

`radix_sort.py` · `busqueda_binaria.py` · `busqueda_hash.py` · `test_algoritmos.py`

Referencias

- Wikipedia. «Radix sort». en.wikipedia.org/wiki/Radix_sort
- Wikipedia. «Binary search». en.wikipedia.org/wiki/Binary_search
- Wikipedia. «Hash table». en.wikipedia.org/wiki/Hash_table

Consultadas el 4 de noviembre de 2026.

¿Preguntas?

Radix Sort

Búsqueda binaria

Búsqueda por hash